

Uni-App 抽离公共方法及抖音小程序条件编译适配指南

在Uni-App跨端开发中，抽离公共方法可提升代码复用性与可维护性，而抖音小程序作为特色平台，存在部分API差异与功能限制。本文将详细讲解公共方法的抽离规范，并结合条件编译实现抖音小程序的精准适配，解决跨端开发中的兼容性问题。

一、核心认知：抖音小程序的适配特殊性

抖音小程序（TikTok Mini Program）基于字节跳动生态，与微信小程序、App等平台相比，在API调用、权限管理、组件支持上存在差异，是跨端适配的重点与难点，主要体现在以下方面：

- API前缀差异**：部分API前缀为`tt`而非`uni`（如抖音原生分享API为`tt.shareAppMessage`，而非`uni.share`）；
- 功能限制**：不支持微信小程序的`wx.getSetting`等部分权限相关API，需使用抖音专属权限接口；
- 组件兼容性**：部分uni-ui组件在抖音小程序中渲染异常，需替换为抖音原生组件或自定义适配；
- 生命周期差异**：页面生命周期与微信小程序基本一致，但应用级生命周期存在细微区别。



Uni-App的条件编译可通过平台标识精准区分抖音小程序，核心标识为`MP-TOUTIAO`（抖音小程序与头条小程序共用该标识，若需单独区分抖音，可结合`tt.getSystemInfo`进一步判断）。

二、公共方法抽离：基础规范与目录结构

公共方法的抽离需遵循“高内聚、低耦合”原则，按功能模块划分，同时预留平台适配入口。以下是标准的目录结构与抽离流程。

1. 推荐目录结构

在项目根目录创建`utils`文件夹，按功能拆分公共方法，其中`platform`子文件夹专门存放平台适配相关逻辑：

Code block

```
1  src/  
2  |—— utils/           // 公共方法根目录
```

```
3 |   |   |   | index.js           // 公共方法入口（统一导出）
4 |   |   |   | common/           // 全平台通用方法
5 |   |   |   | |   |   |   | format.js       // 格式化工具（时间、价格等）
6 |   |   |   | |   |   |   | validate.js     // 校验工具（手机号、邮箱等）
7 |   |   |   | |   |   |   | platform/       // 平台适配方法
8 |   |   |   | |   |   |   | share.js        // 分享功能（含各平台适配）
9 |   |   |   | |   |   |   | auth.js         // 权限获取（含各平台适配）
10 |  |   |   | |   |   |   | pay.js          // 支付功能（含各平台适配）
11 |  |   |   | |   |   |   | pages/          // 业务页面
```

2. 公共方法抽离规范

- **单一职责**：一个公共方法只实现一个核心功能（如`formatTime`仅处理时间格式化）；
- **参数明确**：入参和返回值类型清晰，必要时添加注释说明；
- **异常处理**：包含参数校验与错误捕获，避免影响调用方；
- **统一出口**：通过`utils/index.js`统一导出所有公共方法，简化调用方式。

3. 通用公共方法示例（全平台适用）

以时间格式化工具为例，实现全平台通用的公共方法：

Code block

```
1 // utils/common/format.js
2 /**
3  * 时间格式化方法
4  * @param {Date|Number} time - 时间 (Date对象或时间戳)
5  * @param {String} format - 目标格式 (如'YYYY-MM-DD HH:MM:SS')
6  * @returns {String} 格式化后的时间字符串
7  */
8 export const formatTime = (time, format = 'YYYY-MM-DD HH:MM:SS') => {
9   // 参数校验
10   if (!time) return '';
11   const date = typeof time === 'number' ? new Date(time) : time;
12
13   const o = {
14     'Y+': date.getFullYear(),
15     'M+': date.getMonth() + 1,
16     'D+': date.getDate(),
17     'H+': date.getHours(),
18     'm+': date.getMinutes(),
19     's+': date.getSeconds()
20   };
21
22   // 替换年份
```

```

23     if (/ (Y+) /.test(format)) {
24         format = format.replace(RegExp.$1, (date.getFullYear() + ' ').substr(4 -
RegExp.$1.length));
25     }
26
27     // 替换月、日、时、分、秒
28     for (const k in o) {
29         if (new RegExp(`(${k})`).test(format)) {
30             const str = o[k] + ' ';
31             format = format.replace(RegExp.$1, RegExp.$1.length === 1 ? str :
str.padStart(2, '0'));
32         }
33     }
34
35     return format;
36 };
37
38 // 价格格式化 (保留两位小数)
39 export const formatPrice = (price) => {
40     if (typeof price !== 'number') price = Number(price) || 0;
41     return price.toFixed(2);
42 };

```

4. 统一导出配置

在`utils/index.js`中统一导出所有公共方法，调用时只需引入该文件：

Code block

```

1  // utils/index.js
2  // 导入通用方法
3  import * as format from './common/format';
4  import * as validate from './common/validate';
5
6  // 导入平台适配方法
7  import * as share from './platform/share';
8  import * as auth from './platform/auth';
9  import * as pay from './platform/pay';
10
11 // 统一导出
12 export default {
13     ...format,
14     ...validate,
15     ...share,
16     ...auth,
17     ...pay
18 };

```

5. 页面中调用公共方法

在业务页面中引入`utils/index.js`，即可调用所有公共方法：

Code block

```
1  <script>
2  import utils from '@utils/index.js';
3
4  export default {
5    onLoad() {
6      // 调用通用方法
7      const nowTime = utils.formatTime(new Date());
8      const price = utils.formatPrice(199);
9      console.log('当前时间: ', nowTime);
10     console.log('格式化价格: ', price);
11
12     // 调用平台适配方法
13     utils.shareApp({ title: '测试分享' });
14   }
15 }
16 </script>
```

三、抖音小程序适配核心：条件编译的应用

针对抖音小程序的API差异，通过Uni-App的条件编译语法`#ifdef MP-TOUTIAO`与`#endif`包裹抖音专属逻辑，实现“一套代码，多端适配”。

1. 条件编译基础语法

语法	说明
<code>#ifdef MP-TOUTIAO</code>	仅在抖音/头条小程序中编译
<code>#ifndef MP-TOUTIAO</code>	除抖音/头条小程序外，其他平台均编译
<code>#endif</code>	条件编译结束标记
<code>#ifdef MP-TOUTIAO && appId === 'xxx'</code>	精准匹配指定appId的抖音小程序（区分测试/正式环境）

2. 场景1：分享功能适配（抖音vs微信）

抖音小程序分享API为`tt.shareAppMessage`，与微信的`wx.shareAppMessage`存在差异，通过条件编译实现适配：

Code block

```
1  // utils/platform/share.js
2  /**
3   * 小程序分享方法（多平台适配）
4   * @param {Object} options - 分享配置 (title、imageUrl等)
5   * @returns {Promise} 分享结果
6   */
7  export const shareApp = (options = {}) => {
8    const defaultOptions = {
9      title: '默认分享标题',
10     imageUrl: '/static/share-icon.png',
11     path: '/pages/index/index'
12   };
13
14   const shareOptions = { ...defaultOptions, ...options };
15
16   return new Promise((resolve, reject) => {
17     // 抖音/头条小程序
18     #ifdef MP-TOUTIAO
19     tt.shareAppMessage({
20       title: shareOptions.title,
21       imageUrl: shareOptions.imageUrl,
22       path: shareOptions.path,
23       success: (res) => {
24         console.log('抖音分享成功: ', res);
25         resolve(res);
26       },
27       fail: (err) => {
28         console.error('抖音分享失败: ', err);
29         reject(err);
30       }
31     });
32     #endif
33
34     // 微信小程序
35     #ifdef MP-WEIXIN
36     wx.shareAppMessage({
37       title: shareOptions.title,
38       imageUrl: shareOptions.imageUrl,
39       path: shareOptions.path,
40       success: (res) => resolve(res),
41       fail: (err) => reject(err)
42     });
```

```

43     #endif
44
45     // App端
46     #ifdef APP-PLUS
47     uni.share({
48         provider: 'weixin',
49         type: 0,
50         title: shareOptions.title,
51         imageUrl: shareOptions.imageUrl,
52         path: shareOptions.path,
53         success: (res) => resolve(res),
54         fail: (err) => reject(err)
55     });
56     #endif
57 });
58 };
59
60 // 抖音小程序专属：分享到抖音好友
61 export const shareToTiktokFriend = (options) => {
62     #ifdef MP-TOUTIAO
63     return new Promise((resolve, reject) => {
64         tt.shareAppMessage({
65             ...options,
66             channel: 'video', // 分享到视频渠道
67             success: (res) => resolve(res),
68             fail: (err) => reject(err)
69         });
70     });
71     #endif
72
73     // 非抖音平台提示不支持
74     #ifndef MP-TOUTIAO
75     return Promise.reject({ errMsg: '该功能仅支持抖音小程序' });
76     #endif
77 };

```

3. 场景2：权限获取适配（抖音授权逻辑）

抖音小程序获取用户信息需通过`tt.getUserProfile`，且权限申请逻辑与微信不同，适配代码如下：

Code block

```

1  // utils/platform/auth.js
2  /**
3   * 获取用户信息（多平台适配）
4   * @returns {Promise} 用户信息
5   */

```

```
6 export const getUserInfo = () => {
7   return new Promise((resolve, reject) => {
8     // 抖音/头条小程序
9     #ifdef MP-TOUTIAO
10    // 先判断是否已授权
11    tt.getSetting({
12      success: (settingRes) => {
13        // 已授权, 直接获取用户信息
14        if (settingRes.authSetting['scope.userInfo']) {
15          tt.getUserInfo({
16            success: (userRes) => resolve(userRes.userInfo),
17            fail: (err) => reject(err)
18          });
19        } else {
20          // 未授权, 引导用户授权 (抖音需通过按钮触发)
21          reject({
22            errMsg: '未授权用户信息',
23            needAuth: true
24          });
25        }
26      },
27      fail: (err) => reject(err)
28    });
29    #endif
30
31    // 微信小程序
32    #ifdef MP-WEIXIN
33    wx.getUserProfile({
34      desc: '用于完善会员资料',
35      success: (res) => resolve(res.userInfo),
36      fail: (err) => reject(err)
37    });
38    #endif
39  });
40 };
41
42 // 抖音小程序专属: 授权按钮回调处理
43 export const handleTiktokAuth = () => {
44   #ifdef MP-TOUTIAO
45   return new Promise((resolve, reject) => {
46     tt.getUserProfile({
47       desc: '获取您的个人信息以完成登录',
48       success: (res) => resolve(res.userInfo),
49       fail: (err) => reject(err)
50     });
51   });
52   #endif
```

4. 场景3：页面样式适配（抖音专属样式）

抖音小程序的导航栏、底部安全区与其他平台存在差异，通过条件编译实现样式适配：

Code block

```
1  <template>
2    <view class="tiktok-adapt-page">
3      <view class="content">
4        抖音小程序样式适配示例
5      </view>
6
7
8      #ifdef MP-TOUTIAO
9        <view class="tiktok-float-btn" @click="handleFloatClick">
10         抖音专属按钮
11        </view>
12      #endif
13    </view>
14  </template>
15
16  <style scoped lang="scss">
17    .content {
18      padding: 20rpx;
19    }
20
21    // 抖音小程序专属样式
22    #ifdef MP-TOUTIAO
23    .tiktok-float-btn {
24      position: fixed;
25      bottom: 80rpx; // 适配抖音底部安全区
26      right: 20rpx;
27      width: 120rpx;
28      height: 120rpx;
29      line-height: 120rpx;
30      text-align: center;
31      background-color: #00c850; // 抖音绿
32      color: white;
33      border-radius: 50%;
34      box-shadow: 0 4rpx 10rpx rgba(0, 0, 0, 0.2);
35    }
36    #endif
37
38    // 微信小程序样式
39    #ifdef MP-WEIXIN
```



```
40 .content {
41   background-color: #f5f5f5;
42 }
43 #endif
44 </style>
```

5. 场景4：API能力降级适配（抖音不支持功能）

抖音小程序不支持微信的`wx.openBluetoothAdapter`等API，需通过条件编译实现功能降级：

Code block

```
1  // utils/platform/device.js
2  /**
3   * 打开蓝牙适配器（多平台适配，抖音降级处理）
4   * @returns {Promise} 操作结果
5   */
6  export const openBluetooth = () => {
7    return new Promise((resolve, reject) => {
8      // 抖音小程序：不支持蓝牙，返回降级提示
9      #ifdef MP-TOUTIAO
10     uni.showToast({
11       title: '抖音小程序暂不支持蓝牙功能',
12       icon: 'none',
13       duration: 2000
14     });
15     reject({ errMsg: '抖音小程序不支持蓝牙', isUnsupported: true });
16     #endif
17
18     // 微信小程序/App端：正常打开蓝牙
19     #ifndef MP-TOUTIAO
20     uni.openBluetoothAdapter({
21       success: (res) => resolve(res),
22       fail: (err) => {
23         if (err.errCode === 10001) {
24           uni.showToast({ title: '蓝牙未开启，请先开启蓝牙', icon: 'none' });
25         }
26         reject(err);
27       }
28     });
29     #endif
30   });
31   };
```

四、进阶技巧：抖音小程序适配强化

1. 区分抖音与头条小程序

`MP-TOUTIAO` 标识同时包含抖音与头条小程序，若需单独适配抖音，可通过`tt.getSystemInfo`获取平台信息进一步区分：

Code block

```
1  // utils/platform/helper.js
2  /**
3   * 判断是否为抖音小程序（区分头条小程序）
4   * @returns {Boolean} 是否为抖音小程序
5   */
6  export const isTiktokMiniProgram = () => {
7    #ifdef MP-TOUTIAO
8      const systemInfo = tt.getSystemInfoSync();
9      // 抖音小程序的platform为'tt'，头条小程序为'toutiao'
10     return systemInfo.platform === 'tt';
11   #endif
12   return false;
13 };
14
15 // 调用示例
16 export const testPlatform = () => {
17   #ifdef MP-TOUTIAO
18     if (isTiktokMiniProgram()) {
19       console.log('当前是抖音小程序');
20       // 抖音专属逻辑
21     } else {
22       console.log('当前是头条小程序');
23       // 头条专属逻辑
24     }
25   #endif
26   };
```

2. 抖音小程序API版本兼容

抖音小程序API版本更新较快，部分新API存在兼容性问题，可通过`tt.canIUse`判断API是否可用：

Code block

```
1  // 抖音小程序：判断是否支持tt.getLocation的type参数
2  export const getTiktokLocation = () => {
3    #ifdef MP-TOUTIAO
4      return new Promise((resolve, reject) => {
5        // 判断API是否支持
6        if (tt.canIUse('getLocation.type')) {
7          tt.getLocation({
```

```

8         type: 'gcj02', // 新API支持的参数
9         success: (res) => resolve(res),
10        fail: (err) => reject(err)
11    });
12    } else {
13        // 兼容旧版本API
14        tt.getLocation({
15            success: (res) => resolve(res),
16            fail: (err) => reject(err)
17        });
18    }
19    });
20    #endif
21 };

```

3. 公共方法的抖音环境初始化

抖音小程序启动时需初始化部分配置（如设置导航栏标题颜色），可在公共方法中封装初始化逻辑，在App.vue中调用：

Code block

```

1  // utils/platform/init.js
2  /**
3   * 抖音小程序初始化配置
4   */
5  export const initTiktokConfig = () => {
6      #ifdef MP-TOUTIAO
7          // 设置导航栏标题颜色
8          tt.setNavigationBarColor({
9              frontColor: '#000000',
10             backgroundColor: '#ffffff'
11         });
12
13         // 初始化抖音统计
14         if (tt.uma) {
15             tt.uma.init({
16                 appKey: 'your-tiktok-uma-key'
17             });
18         }
19
20         console.log('抖音小程序初始化完成');
21         #endif
22     };
23
24     // App.vue中调用
25     <script>

```

```

26 import utils from '@utils/index.js';
27
28 export default {
29   onLaunch() {
30     // 平台初始化
31     #ifdef MP-TOUTIAO
32       utils.initTiktokConfig();
33     #endif
34   }
35 }
36 </script>

```

4. 错误处理与日志上报（抖音专属）

抖音小程序提供专属的日志上报API，可在公共方法中封装错误处理逻辑：

Code block

```

1  // utils/platform/error.js
2  /**
3   * 错误处理与日志上报（多平台适配）
4   * @param {Object} err - 错误信息
5   * @param {String} type - 错误类型
6   */
7  export const handleError = (err, type = 'common') => {
8    console.error(`[${type}错误]`, err);
9
10   // 抖音小程序：上报错误到抖音日志系统
11   #ifdef MP-TOUTIAO
12   if (tt.reportMonitor) {
13     tt.reportMonitor({
14       name: `${type}_error`,
15       value: 1,
16       dimension: {
17         errMsg: err.errMsg || JSON.stringify(err),
18         page: getCurrentPages().pop().route
19       }
20     });
21   }
22   #endif
23
24   // 微信小程序：上报到微信日志
25   #ifdef MP-WEIXIN
26   wx.reportMonitor('error_count', 1);
27   #endif
28 };

```

五、注意事项与最佳实践

1. 条件编译使用规范

- **避免过度嵌套**：条件编译嵌套层级不超过2层，防止代码可读性下降；
- **统一平台标识**：全项目统一使用`MP-TOUTIAO`标识抖音/头条小程序，避免混用自定义判断；
- **残留代码清理**：上线前删除条件编译块中的测试代码，减少包体积。

2. 抖音小程序权限申请规范

- **授权前说明**：申请用户信息、地理位置等权限前，需通过弹窗说明授权用途，提升用户同意率；
- **拒绝后处理**：用户拒绝授权后，提供“去设置”按钮引导用户手动开启权限：

Code block

```
1  // 抖音小程序：引导用户去设置权限
2  export const guideToSetting = () => {
3    #ifdef MP-TOUTIAO
4    uni.showModal({
5      title: '权限申请',
6      content: '该功能需要获取您的位置信息，请前往设置开启',
7      confirmText: '去设置',
8      success: (res) => {
9        if (res.confirm) {
10          tt.openSetting({
11            success: (settingRes) => {
12              console.log('设置页面返回:', settingRes);
13            }
14          });
15        }
16      }
17    });
18    #endif
19  };
```

3. 性能优化建议

- **按需引入**：若项目体积较大，可通过ES模块按需引入公共方法，而非全量导入；
- **缓存重复数据**：抖音小程序的系统信息、用户授权状态等可缓存到全局，避免重复调用API；
- **避免同步阻塞**：抖音小程序中`tt.getSystemInfoSync`等同步API尽量在启动时调用，避免在页面渲染中使用。

4. 测试与调试技巧

- **使用抖音开发者工具：**通过抖音小程序开发者工具调试专属逻辑，避免依赖HBuilderX模拟器；
- **真机测试：**抖音小程序的部分API（如分享）仅在真机上可用，需重点进行真机测试；
- **版本兼容测试：**使用抖音不同版本的手机测试，确保低版本抖音也能正常运行。

六、总结：跨端适配的核心思路

Uni-App开发中，抽离公共方法与抖音小程序适配的核心思路是“共性抽离，个性适配”：

1. **共性抽离：**将时间格式化、数据校验等全平台通用逻辑抽离为公共方法，减少代码冗余；
2. **个性适配：**针对抖音小程序的API差异，通过条件编译实现专属逻辑，不影响其他平台；
3. **异常处理：**为各平台适配逻辑添加完善的错误处理，提升应用稳定性；
4. **规范约束：**制定公共方法命名、参数传递的统一规范，便于团队协作。



随着抖音小程序生态的不断完善，API与功能会持续更新，开发中需定期关注抖音小程序官方文档，及时适配新特性与变更，确保应用的兼容性与功能完整性。

（注：文档部分内容可能由 AI 生成）